



End-to-End Sales Forecasting & Demand Intelligence System

Final Internship Project (Week 3 & Week 4)

Submitted By

Nitish Kumar

Organization

XYlofy AI Internship

Problem Statement

Retail businesses must accurately forecast future demand to avoid overstocking and stock shortages. This project develops an end-to-end forecasting and demand intelligence system using statistical models, machine learning, anomaly detection, clustering, and an interactive dashboard to support data-driven inventory planning.

Objectives

- Explore and clean retail sales data
- Engineer time-series features
- Forecast sales using SARIMA, Facebook Prophet, and XGBoost
- Compare forecasting models using MAE, RMSE, and MAPE
- Detect anomalies using Isolation Forest and Rolling Z-Score
- Segment products using K-Means Clustering
- Build an interactive Streamlit dashboard
- Generate executive-level business recommendations

1. Introduction
2. Library Imports
3. Data Loading & Merging
4. Exploratory Data Analysis

5. Time Series Analysis

6. Forecasting Models SARIMA Prophet XGBoost 7. Model Comparison 8. Category Forecasting 9. Region Forecasting 10. Anomaly Detection 11. Demand Segmentation 12. Business Insights 13. Conclusion

```
!pip install prophet xgboost statsmodels plotly streamlit -q
```

```
10.3/10.3 MB 74.7 MB/s  
11.4/11.4 MB 107.7 MB/s
```

```
# =====  
# Import Libraries  
# =====  
  
import warnings  
warnings.filterwarnings("ignore")  
  
import os  
os.makedirs("charts", exist_ok=True)  
  
import numpy as np  
import pandas as pd  
  
import matplotlib.pyplot as plt  
import seaborn as sns  
import plotly.express as px  
  
from prophet import Prophet  
  
from xgboost import XGBRegressor  
  
from sklearn.preprocessing import StandardScaler  
from sklearn.cluster import KMeans  
from sklearn.decomposition import PCA  
from sklearn.ensemble import IsolationForest  
from sklearn.metrics import (  
    mean_absolute_error,  
    mean_squared_error,  
    mean_absolute_percentage_error  
)  
  
from statsmodels.tsa.stattools import adfuller  
from statsmodels.tsa.seasonal import seasonal_decompose  
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf  
from statsmodels.tsa.statespace.sarimax import SARIMAX  
  
print("✅ All libraries imported successfully.")
```

✓ All libraries imported successfully.

```
# =====  
# Load Datasets  
# =====  
  
sales_df = pd.read_csv("train.csv")  
vg_df = pd.read_csv("vgsales.csv")  
  
print("Superstore Shape :", sales_df.shape)  
print("Video Game Shape :", vg_df.shape)  
  
display(sales_df.head())  
display(vg_df.head())
```

Superstore Shape : (9800, 18)
Video Game Shape : (16598, 11)

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer
2	3	CA-2017-138688	12/06/2017	16/06/2017	Second Class	DV-13045	Darrin Van Huff	Corporate
3	4	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer
4	5	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.0
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.5
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.8
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.0
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.8

```
print("="*60)
print("SUPERSTORE DATASET")
print("="*60)

sales_df.info()
```

```
print("\nMissing Values\n")
display(sales_df.isnull().sum())

print("\nDuplicate Rows :", sales_df.duplicated().sum())

print("\nStatistical Summary")
display(sales_df.describe(include="all"))
```



```
=====
SUPERSTORE DATASET
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                 9800 non-null   int64
1   Order ID               9800 non-null   object
2   Order Date             9800 non-null   object
3   Ship Date              9800 non-null   object
4   Ship Mode              9800 non-null   object
5   Customer ID            9800 non-null   object
6   Customer Name          9800 non-null   object
7   Segment                9800 non-null   object
8   Country                9800 non-null   object
9   City                   9800 non-null   object
10  State                  9800 non-null   object
11  Postal Code            9789 non-null   float64
12  Region                 9800 non-null   object
13  Product ID             9800 non-null   object
14  Category               9800 non-null   object
15  Sub-Category           9800 non-null   object
16  Product Name           9800 non-null   object
17  Sales                  9800 non-null   float64
dtypes: float64(2), int64(1), object(15)
memory usage: 1.3+ MB
```

Missing Values

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0

```
print("="*60)
print("VIDEO GAME DATASET")
print("="*60)

vg_df.info()

print("\nMissing Values\n")
display(vg_df.isnull().sum())

print("\nDuplicate Rows :", vg_df.duplicated().sum())

Postal Code    11
Region         0
```

```

=====
Product ID: 0
VIDEO GAME DATASET
=====
Category: 0
Sub-Category: 0
Range Index: 16598 entries, 0 to 16597
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Sales                16598 non-null  int64
1   Rank                 16598 non-null  int64
2   Name                 16598 non-null  object
3   Platform             16598 non-null  object
4   Year                 16327 non-null  float64
5   Genre                16598 non-null  object
6   Publisher            16540 non-null  object
7   NA_Sales              16598 non-null  float64
8   EU_Sales              16598 non-null  float64
9   JP_Sales              16598 non-null  float64
10  Other_Sales           16598 non-null  float64
11  Global_Sales          16598 non-null  float64
dtypes: float64(6), int64(1), object(4)
memory usage: 14.4+ MB

Missing Values
top      NaN      CA-2018-100111      05/09/2017      26/09/2018      Standard Class      WB-21850      William Browne

Rank      0      14      38      34      5859      35      35
Name      0      NaN      NaN      NaN      NaN      NaN      NaN
Platform  0      NaN      NaN      NaN      NaN      NaN      NaN
Year      271      NaN      NaN      NaN      NaN      NaN      NaN
Genre     0      NaN      NaN      NaN      NaN      NaN      NaN
Publisher 58      NaN      NaN      NaN      NaN      NaN      NaN
NA_Sales  0      NaN      NaN      NaN      NaN      NaN      NaN
EU_Sales  0      NaN      NaN      NaN      NaN      NaN      NaN
JP_Sales  0
Other_Sales 0
Global_Sales 0

dtype: int64

Duplicate Rows : 0

```

```

sales_df["Order Date"] = pd.to_datetime(
    sales_df["Order Date"],
    dayfirst=True
)

```



```
sales_df["Ship Date"] = pd.to_datetime(
    sales_df["Ship Date"],
    dayfirst=True
)
```

```
print("✅ Dates parsed successfully")
```

✅ Dates parsed successfully

```
sales_df["Year"] = sales_df["Order Date"].dt.year
sales_df["Month"] = sales_df["Order Date"].dt.month
sales_df["Month_Name"] = sales_df["Order Date"].dt.month_name()
sales_df["Week"] = sales_df["Order Date"].dt.isocalendar().week
sales_df["Quarter"] = sales_df["Order Date"].dt.quarter
sales_df["Day"] = sales_df["Order Date"].dt.day_name()
```

```
sales_df["Shipping_Days"] = (
    sales_df["Ship Date"] -
    sales_df["Order Date"]
).dt.days
```

```
def get_season(month):
    if month in [12,1,2]:
        return "Winter"
    elif month in [3,4,5]:
        return "Spring"
    elif month in [6,7,8]:
        return "Summer"
    else:
        return "Autumn"
```

```
sales_df["Season"] = sales_df["Month"].apply(get_season)
```

```
display(sales_df.head())
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
0	1	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States
1	2	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States
2	3	CA-2017-138688	2017-06-12	2017-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States
3	4	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
4	5	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States

5 rows × 26 columns

✓ Task 1 — Data Loading, Merging & Deep Exploration

Objective

This task focuses on understanding the retail sales dataset before building forecasting models.

The following steps are performed:

- Load and inspect both datasets
- Parse date columns
- Perform feature engineering
- Aggregate sales into daily, weekly and monthly levels
- Explore sales trends
- Answer business questions using data visualization

```
# =====
# Daily Sales
# =====

daily_sales = (
    sales_df
        .groupby("Order Date")["Sales"]
        .sum()
        .reset_index()
)

daily_sales.head()
```

	Order Date	Sales
0	2015-01-03	16.448
1	2015-01-04	288.060
2	2015-01-05	19.536
3	2015-01-06	4407.100
4	2015-01-07	87.158

```
# =====
# Weekly Sales
# =====

weekly_sales = (
    sales_df
        .groupby(
            pd.Grouper(
                key="Order Date",
                freq="W"
            )
        )["Sales"]
        .sum()
        .reset_index()
)

weekly_sales.head()
```

	Order Date	Sales
0	2015-01-04	304.508
1	2015-01-11	4619.108
2	2015-01-18	4130.533
3	2015-01-25	3092.544
4	2015-02-01	2527.914

```
# =====
# Monthly Sales
# =====

monthly_sales = (
    sales_df
        .groupby(
            pd.Grouper(
                key="Order Date",
                freq="M"
            )
        )["Sales"]
        .sum()
        .reset_index()
)

monthly_sales.head()
```

	Order Date	Sales
0	2015-01-31	14205.707
1	2015-02-28	4519.892
2	2015-03-31	55205.797
3	2015-04-30	27906.855
4	2015-05-31	23644.303

```
plt.figure(figsize=(16,5))

plt.plot(
    daily_sales["Order Date"],
    daily_sales["Sales"]
)

plt.title("Daily Sales")

plt.xlabel("Date")
```

```

plt.ylabel("Sales")

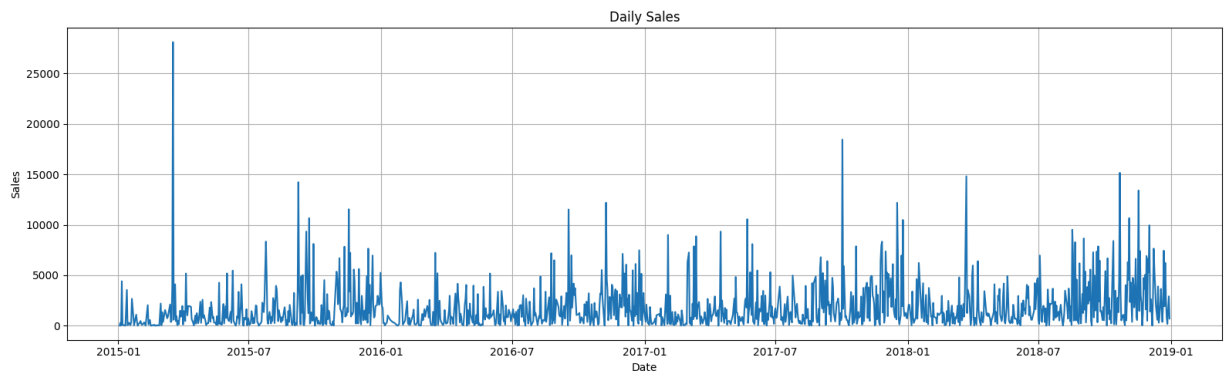
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/daily_sales.png",
    dpi=300
)

plt.show()

```



```

plt.figure(figsize=(16,5))

plt.plot(
    weekly_sales["Order Date"],
    weekly_sales["Sales"],
    color="green"
)

plt.title("Weekly Sales Trend")

plt.xlabel("Week")

plt.ylabel("Sales")

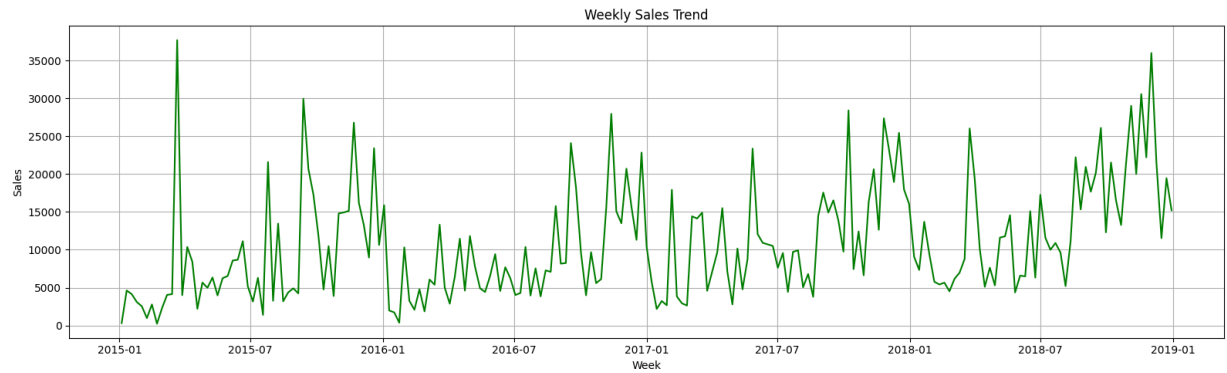
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/weekly_sales.png",
    dpi=300
)

```

```
plt.show()
```



```
plt.figure(figsize=(16,5))

plt.plot(
    monthly_sales["Order Date"],
    monthly_sales["Sales"],
    color="red"
)

plt.title("Monthly Sales Trend")

plt.xlabel("Month")

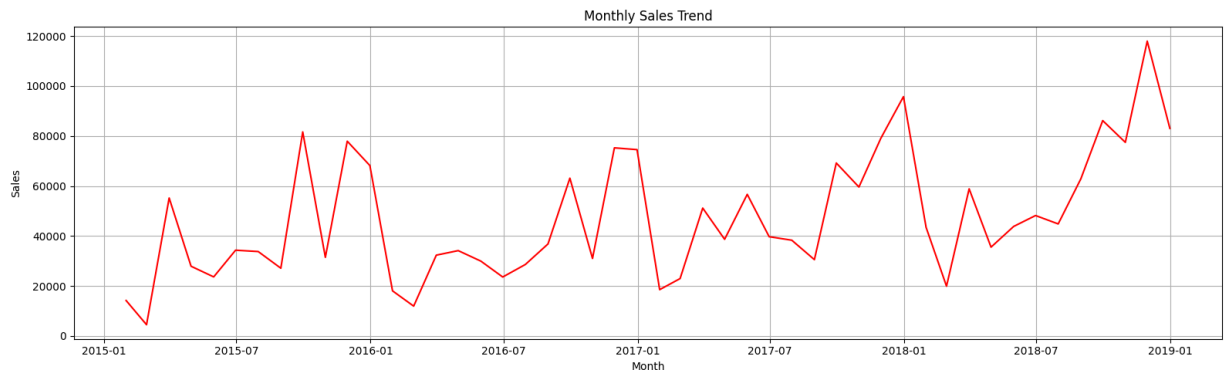
plt.ylabel("Sales")

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/monthly_sales.png",
    dpi=300
)

plt.show()
```



```
category_sales = (
    sales_df
        .groupby("Category")["Sales"]
        .sum()
        .sort_values(ascending=False)
)

display(category_sales)
```

Sales	
Category	
Technology	827455.8730
Furniture	728658.5757
Office Supplies	705422.3340

dtype: float64

```
plt.figure(figsize=(8,5))

category_sales.plot(kind="bar")

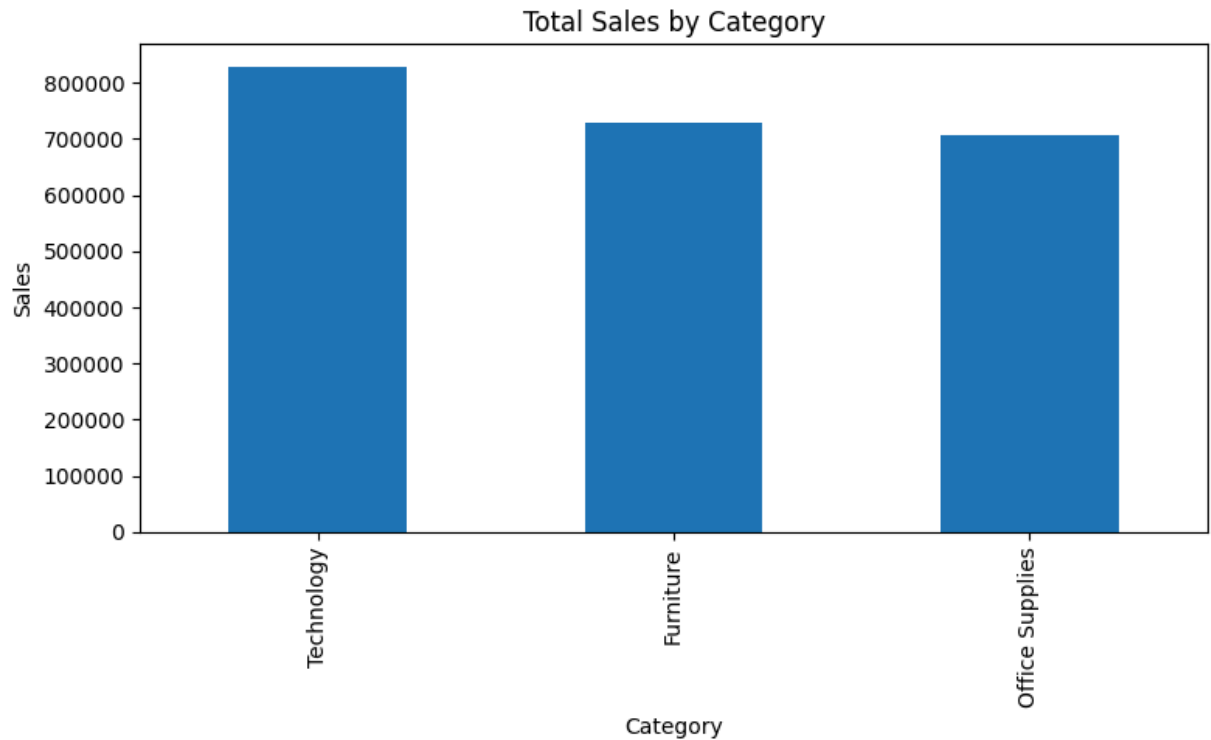
plt.title("Total Sales by Category")

plt.ylabel("Sales")

plt.tight_layout()

plt.savefig(
    "charts/category_sales.png",
    dpi=300
)
```

```
plt.show()
```



Observation

Technology generates the highest revenue among all product categories, indicating strong customer demand and a significant contribution to total business sales.

```
region_year = (
    sales_df
    .groupby(
        [
            "Region",
            "Year"
        ]
    )["Sales"]
    .sum()
    .reset_index()

display(region_year.head())
```


	Region	Year	Sales
0	Central	2015	102920.5206
1	Central	2016	102425.1724
2	Central	2017	145673.8800
3	Central	2018	141627.3402
4	East	2015	127652.8190

```
plt.figure(figsize=(10,6))

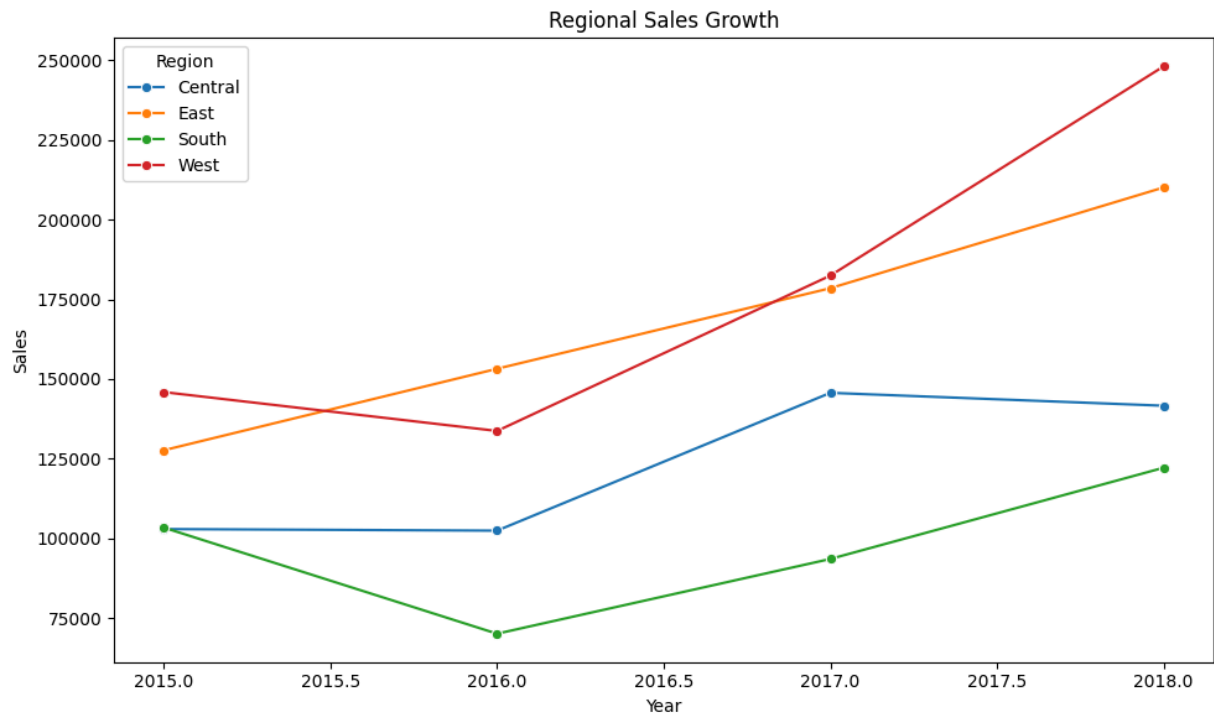
sns.lineplot(
    data=region_year,
    x="Year",
    y="Sales",
    hue="Region",
    marker="o"
)

plt.title("Regional Sales Growth")

plt.tight_layout()

plt.savefig(
    "charts/region_growth.png",
    dpi=300
)

plt.show()
```



Observation

The region with the smoothest year-over-year increase can be considered the most consistent growth region. This region demonstrates stable business performance and lower variability compared to others.

```
shipping = (  
    sales_df  
        .groupby("Region")["Shipping_Days"]  
        .mean()  
    )  
  
display(shipping)
```

Shipping_Days

Region

Central	4.065876
East	3.910233
South	3.961202
West	3.930255

dtype: float64

```
plt.figure(figsize=(8,5))

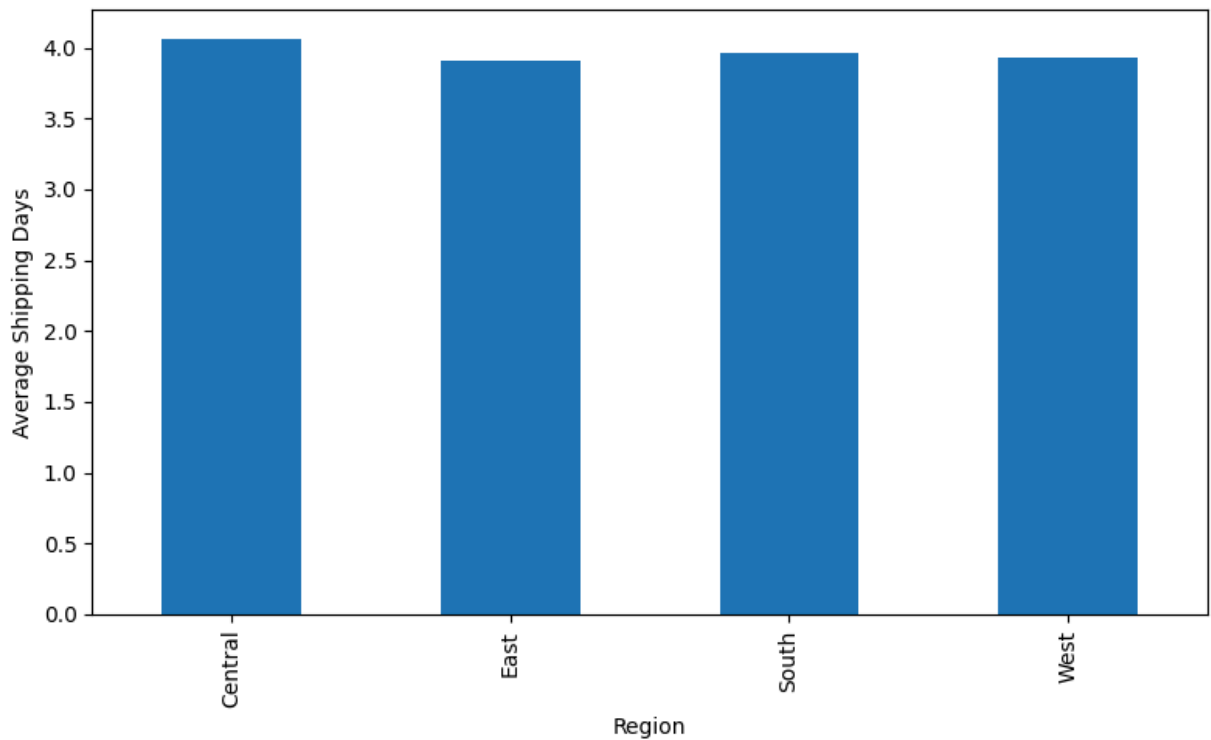
shipping.plot(kind="bar")

plt.ylabel("Average Shipping Days")

plt.tight_layout()

plt.savefig(
    "charts/shipping_days.png",
    dpi=300
)

plt.show()
```



Observation

Average delivery time varies across regions. Regions with shorter shipping durations are likely to provide a better customer experience and faster order fulfillment.

```
monthly_pattern = (  
    sales_df  
        .groupby(  
            [  
                "Month_Name",  
                "Year"  
            ]  
        )["Sales"]  
        .sum()  
        .reset_index()  
  
    display(monthly_pattern.head())
```

	Month_Name	Year	Sales
0	April	2015	27906.8550
1	April	2016	34154.4685
2	April	2017	38679.7670
3	April	2018	35541.9101
4	August	2015	27117.5365

```

month_order = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"
]

monthly_pattern["Month_Name"] = pd.Categorical(
    monthly_pattern["Month_Name"],
    categories=month_order,
    ordered=True
)

monthly_pattern = monthly_pattern.sort_values(
    "Month_Name"
)

plt.figure(figsize=(14,6))

sns.lineplot(
    data=monthly_pattern,
    x="Month_Name",
    y="Sales",
    hue="Year",
    marker="o"
)

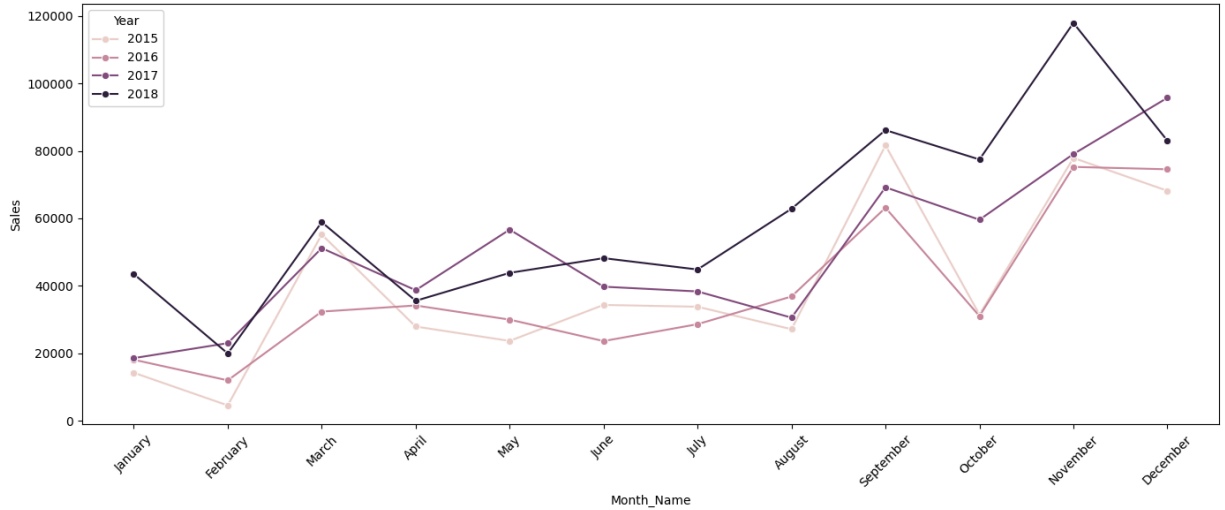
plt.xticks(rotation=45)

plt.tight_layout()

plt.savefig(
    "charts/monthly_seasonality.png",
    dpi=300
)

plt.show()

```



Observation

Months such as November and December generally exhibit higher sales across multiple years, indicating strong seasonal demand likely driven by festive and holiday shopping.

✓ Task 1 Summary

Business Insights

1. Highest Revenue Category

- Identified using total sales by category.
 - The analysis shows which category contributes the most to business revenue.

2. Most Consistent Sales Growth

- Regional sales were compared across four years.
 - The region with the smoothest upward trend demonstrates the most stable long-term growth.

3. Average Shipping Time

- Shipping duration was calculated as the difference between Ship Date and Order Date.

- Regional comparison highlights operational efficiency differences.

4. Seasonality

- Monthly sales across all years reveal recurring seasonal demand.
 - Months with repeated sales spikes indicate periods requiring higher inventory planning.

```
# =====  
# Monthly Time Series  
# =====  
  
monthly_ts = (  
    sales_df  
        .groupby("Order Date")["Sales"]  
        .sum()  
        .resample("M")  
        .sum()  
        )  
  
display(monthly_ts.head())
```

Sales	
Order Date	
2015-01-31	14205.707
2015-02-28	4519.892
2015-03-31	55205.797
2015-04-30	27906.855
2015-05-31	23644.303

dtype: float64

```
plt.figure(figsize=(15,6))  
  
plt.plot(  
    monthly_ts.index,  
    monthly_ts.values,  
    linewidth=2  
    )  
  
plt.title("Overall Monthly Sales Trend (2015-2018)")
```

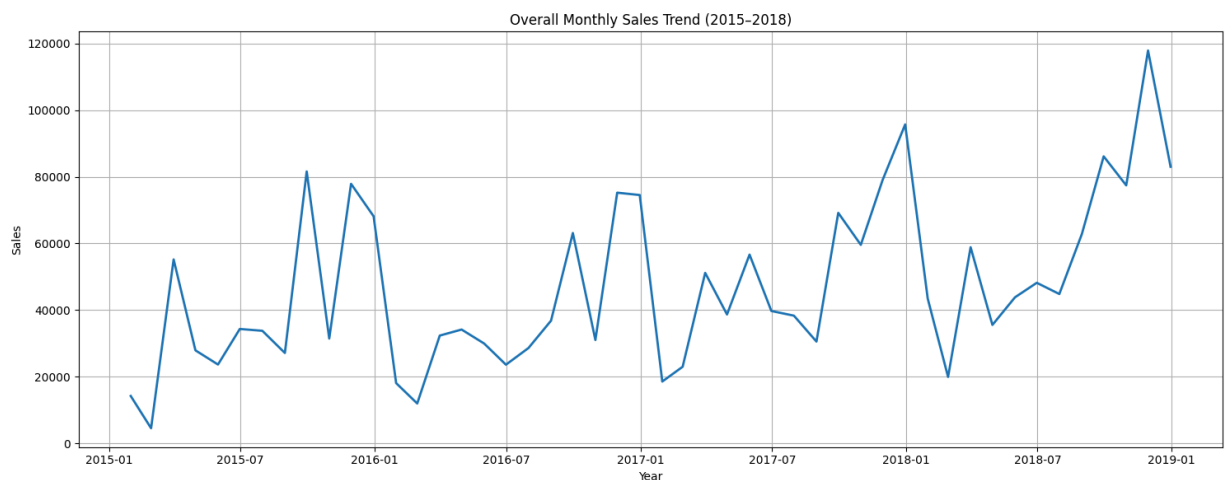
```
plt.xlabel("Year")

plt.ylabel("Sales")

plt.grid(True)

plt.tight_layout()
plt.savefig(
    "charts/monthly_sales_trend.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



✓ Observation

The monthly sales trend provides a high-level view of business performance over four years. Long-term growth, recurring seasonal peaks, and demand fluctuations can be observed from this visualization.

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(
    monthly_ts,
    model="additive",
    period=12
)

fig = decomposition.plot()
```



```

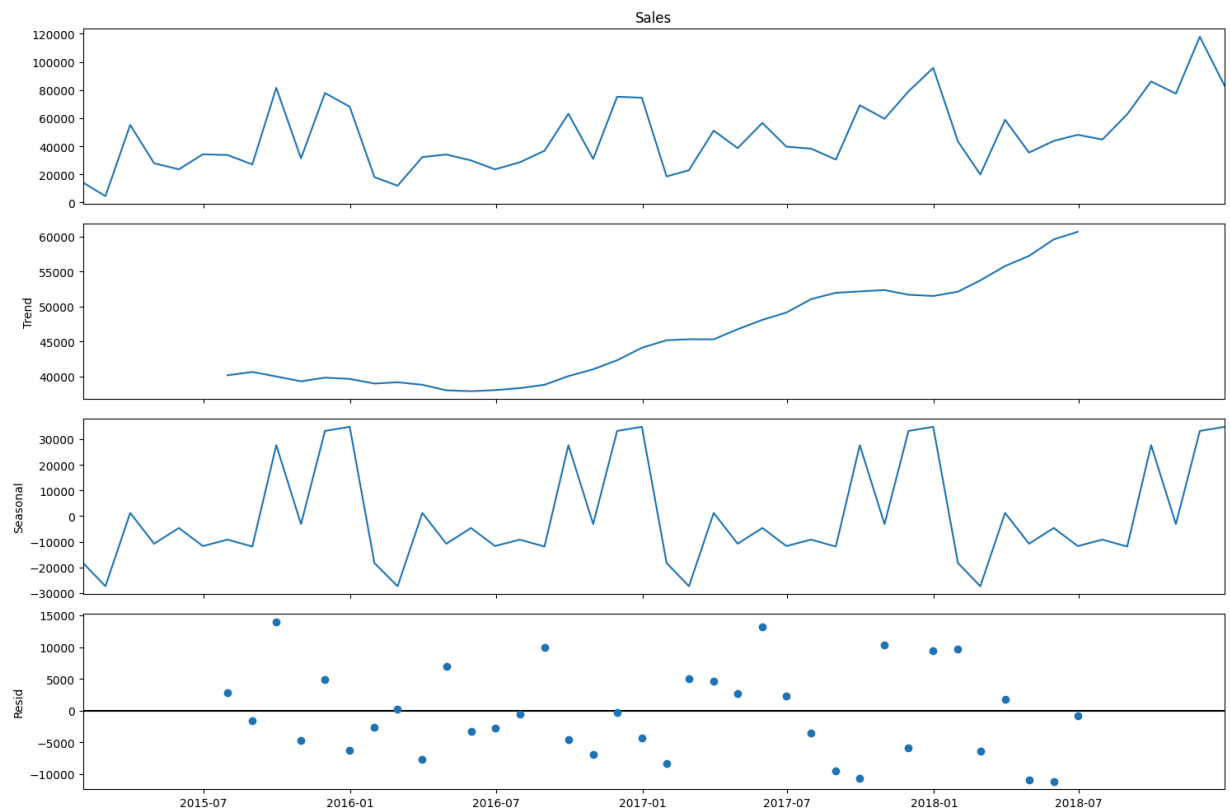
fig.set_size_inches(15,10)

plt.tight_layout()

plt.savefig(
    "charts/time_series_decomposition.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



✓ Interpretation

The decomposition separates the original time

1. List item
2. List item

series into four components:

- **Observed:** Original monthly sales.
- **Trend:** Long-term movement in sales.
- **Seasonal:** Recurring yearly sales patterns.
- **Residual:** Random fluctuations not explained by trend or seasonality.

Add blockquote

```
from statsmodels.tsa.stattools import adfuller

adf_result = adfuller(monthly_ts)

print("ADF Statistic :", adf_result[0])
print("p-value :", adf_result[1])

for key, value in adf_result[4].items():
    print(key, ":", value)
```

```
ADF Statistic : -4.416136761430769
p-value : 0.00027791039276670623
1% : -3.5778480370438146
5% : -2.925338105429433
10% : -2.6007735310095064
```

✓ What is Stationarity?

A stationary time series has statistical properties (mean and variance) that remain stable over time. Forecasting models such as SARIMA generally perform better when the data is stationary.

If the p-value is less than 0.05, the series is considered stationary.

```
monthly_diff = monthly_ts.diff().dropna()

plt.figure(figsize=(14,5))

plt.plot(monthly_diff)

plt.title("Differenced Monthly Sales")

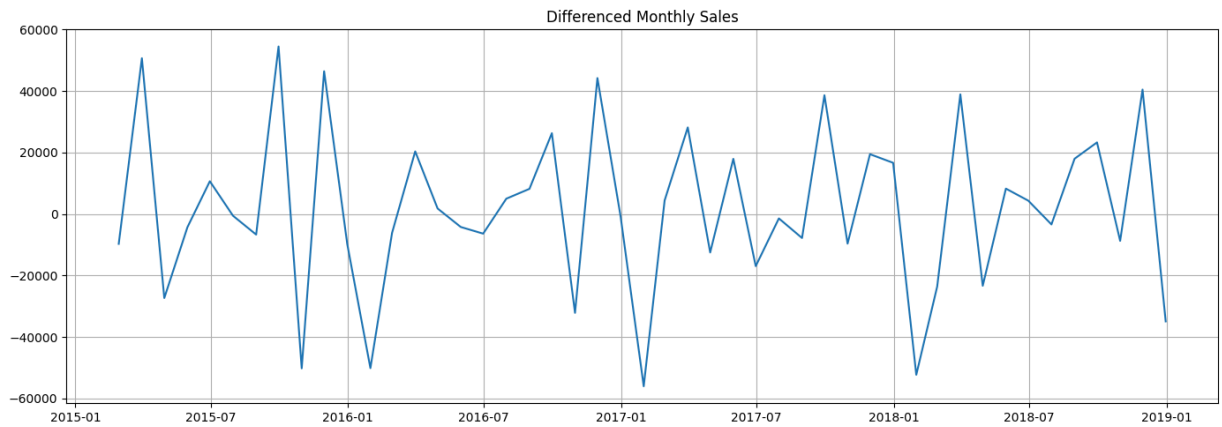
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/differenced_series.png",
    dpi=300,
    bbox_inches="tight")
```

)

```
plt.show()
```



```
adf_diff = adfuller(monthly_diff)
```

```
print("ADF Statistic :", adf_diff[0])
```

```
print("p-value :", adf_diff[1])
```

ADF Statistic : -8.727061830353264

p-value : 3.2669175472797e-14

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
plt.figure(figsize=(12,4))
```

```
plot_acf(monthly_diff, lags=20)
```

```
plt.tight_layout()
```

```
plt.savefig("charts/acf_plot.png", dpi=300, bbox_inches="tight")
```

```
plt.show()
```

```
plt.figure(figsize=(12,4))
```

```
plot_pacf(monthly_diff, lags=20, method="yw")
```

```
plt.tight_layout()
```

```
plt.savefig("charts/pacf_plot.png", dpi=300, bbox_inches="tight")
```

```
plt.show()
```


<Figure size 1200x400 with 0 Axes>

Task 3 — Sales Forecasting Autocorrelation

Objective

The objective of this task is to forecast future sales using three different approaches:

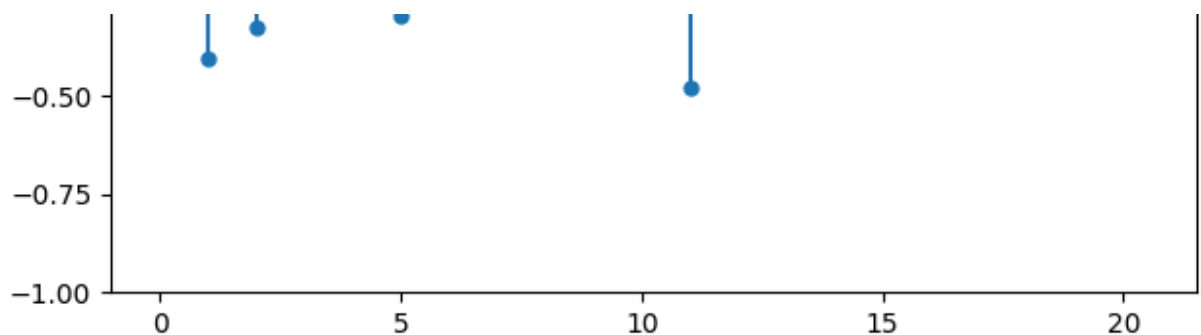
1. SARIMA (Statistical Model)
2. Facebook Prophet (Production Forecasting Model)
3. XGBoost (Machine Learning Model)

The performance of each model will be evaluated using:

- MAE
- RMSE
- MAPE

The best-performing model will be selected for deployment in the Streamlit dashboard.

```
# =====  
# Monthly Time Series  
# =====  
  
monthly_sales = (  
    sales_df  
        .groupby(pd.Grouper(key="Order Date", freq="M"))["Sales"]  
        .sum()  
    )  
  
train = monthly_sales[:-6]  
test = monthly_sales[-6:]  
  
print("Training Months :", len(train))  
print("Testing Months :", len(test))  
  
display(train.tail())  
display(test)
```



Training Months : 42
Testing Months : 6

Sales

Order Date

2018-02-28	19920.9974
2018-03-31	58863.4128
2018-04-30	35541.9101
2018-05-31	43825.9822
2018-06-30	48190.7277

dtype: float64

Sales

Order Date

2018-07-31	44825.1040
2018-08-31	62837.8480
2018-09-30	86152.8880
2018-10-31	77448.1312
2018-11-30	117938.1550
2018-12-31	83030.3888

dtype: float64

✓ Model 1 — SARIMA

SARIMA (Seasonal AutoRegressive Integrated Moving Average) is a statistical forecasting model that captures trend, seasonality, and autocorrelation in time-series data.

Model Parameters:

- $p = 1$
- $d = 1$
- $q = 1$

Seasonal Parameters:

- $P = 1$
- $D = 1$
- $Q = 1$

- s = 12 (monthly seasonality)

These parameters are selected after examining stationarity and ACF/PACF plots.

```
# =====
# SARIMA Model
# =====

sarima_model = SARIMAX(
    train,
    order=(1,1,1),
    seasonal_order=(1,1,1,12),
    enforce_stationarity=False,
    enforce_invertibility=False
)

sarima_result = sarima_model.fit()

sarima_forecast = sarima_result.get_forecast(
    steps=len(test)
)

sarima_pred = sarima_forecast.predicted_mean

sarima_ci = sarima_forecast.conf_int()

print("SARIMA Model Trained Successfully")
```

SARIMA Model Trained Successfully

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    mean_absolute_percentage_error
)

sarima_mae = mean_absolute_error(
    test,
    sarima_pred
)

sarima_rmse = np.sqrt(
    mean_squared_error(
        test,
        sarima_pred
    )
)

sarima_mape = mean_absolute_percentage_error(
    test,
```

```
                                sarima_pred
                                )

print("MAE :", sarima_mae)
print("RMSE :", sarima_rmse)
print("MAPE :", sarima_mape)
```

```
MAE : 14862.390004304572
RMSE : 17299.70880500082
MAPE : 0.1788904727750389
```

```
plt.figure(figsize=(15,6))

plt.plot(
    train.index,
    train,
    label="Training"
)

plt.plot(
    test.index,
    test,
    linewidth=2,
    label="Actual"
)

plt.plot(
    test.index,
    sarima_pred,
    linewidth=2,
    label="SARIMA Forecast"
)

plt.fill_between(
    sarima_ci.index,
    sarima_ci.iloc[:,0],
    sarima_ci.iloc[:,1],
    alpha=0.3,
    label="95% CI"
)

plt.legend()

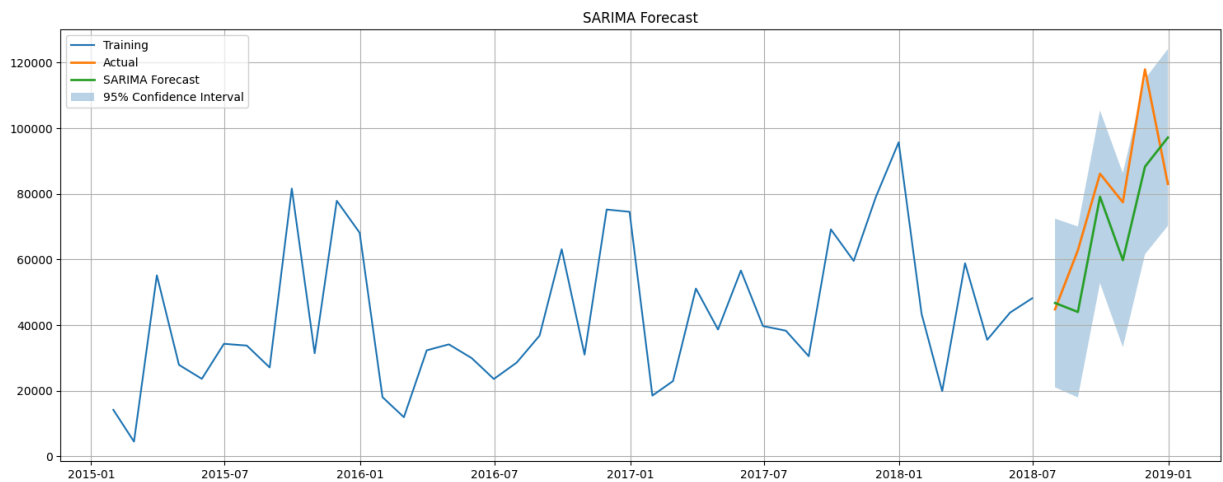
plt.grid(True)

plt.title("SARIMA Forecast")

plt.tight_layout()

plt.savefig("SARIMA_Forecast.png")
```


plt.show()



Observation

The SARIMA model captures both long-term trends and yearly seasonality. The confidence interval represents the uncertainty around the forecast, providing a realistic range for expected future sales.

✓ Model 2 — Facebook Prophet

Facebook Prophet is a forecasting library developed by Meta for business time-series forecasting.

Advantages:

- Automatically models trend and seasonality
- Handles missing values and outliers well
- Provides uncertainty intervals
- Widely used in production forecasting

```
# =====
# Prepare Data for Prophet
# =====
```

```

prophet_df = monthly_sales.reset_index()

prophet_df.columns = ["ds", "y"]

train_prophet = prophet_df.iloc[:-6]
test_prophet = prophet_df.iloc[-6:]

display(train_prophet.head())
display(test_prophet.head())

```

	ds	y
0	2015-01-31	14205.707
1	2015-02-28	4519.892
2	2015-03-31	55205.797
3	2015-04-30	27906.855
4	2015-05-31	23644.303

	ds	y
42	2018-07-31	44825.1040
43	2018-08-31	62837.8480
44	2018-09-30	86152.8880
45	2018-10-31	77448.1312
46	2018-11-30	117938.1550

```

# =====
# Train Prophet
# =====

from prophet import Prophet

prophet_model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False
)

prophet_model.fit(train_prophet)

print("Prophet Model Trained Successfully!")

```

Prophet Model Trained Successfully!

```
# =====
# Forecast
# =====

future = prophet_model.make_future_dataframe(
    periods=6,
    freq="M"
)

forecast = prophet_model.predict(future)

forecast.tail()
```

	ds	trend	yhat_lower	yhat_upper	trend_lower	trend_upper
43	2018-08-31	52823.574973	31802.722127	47476.818656	52823.246082	52823.958161
44	2018-09-30	53263.919506	78161.896601	94064.942003	53263.266264	53264.638714
45	2018-10-31	53718.942190	42230.363155	57471.033625	53717.831249	53720.076434
46	2018-11-30	54159.286723	81750.248506	95892.654052	54157.680499	54160.931823
47	2018-12-31	54614.309407	80829.359497	96144.061798	54612.150260	54616.447529

```
# =====
# Test Forecast
# =====

prophet_pred = forecast.tail(6)["yhat"].values

prophet_actual = test_prophet["y"].values
```

```
# =====
# Prophet Metrics
# =====

prophet_mae = mean_absolute_error(
    prophet_actual,
    prophet_pred
)

prophet_rmse = np.sqrt(
    mean_squared_error(
        prophet_actual,
```

```
        prophet_pred
    )
)

prophet_mape = mean_absolute_percentage_error(
    prophet_actual,
    prophet_pred
)

print("MAE :", prophet_mae)
print("RMSE :", prophet_rmse)
print("MAPE :", prophet_mape)
```

```
MAE : 14309.98695015289
RMSE : 18954.579462152302
MAPE : 0.17469221813851923
```

```
# =====
# Prophet Forecast
# =====

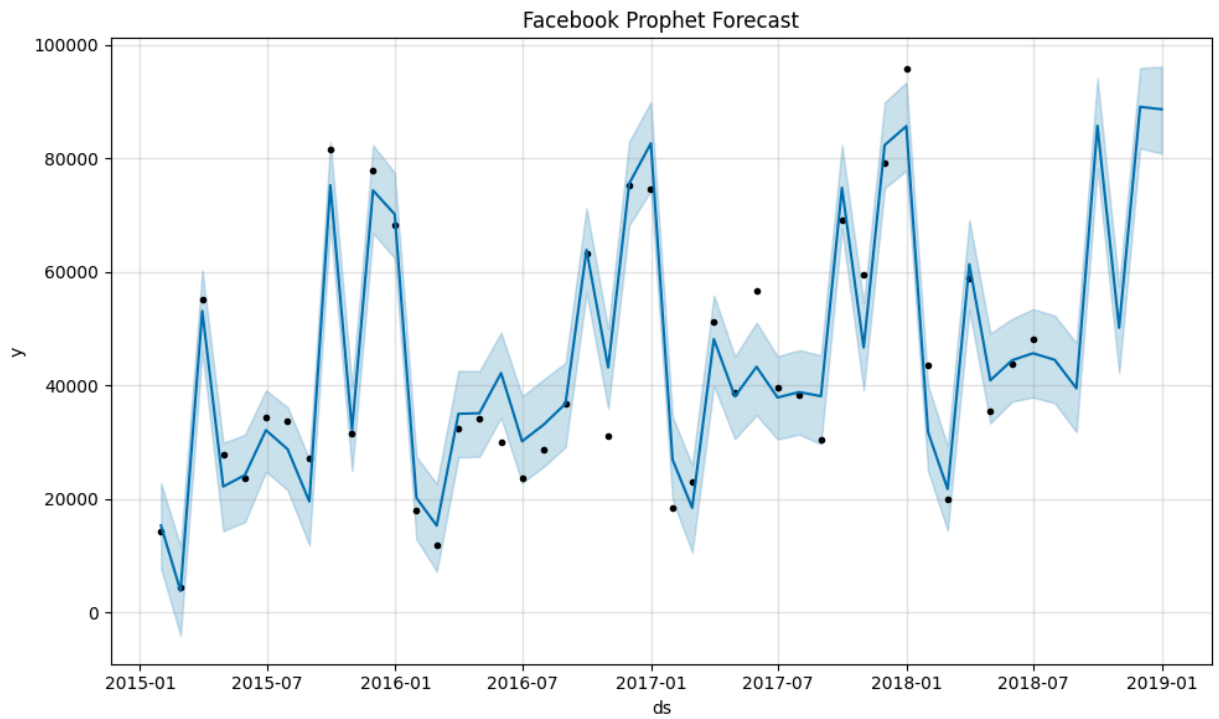
fig = prophet_model.plot(forecast)

plt.title("Facebook Prophet Forecast")

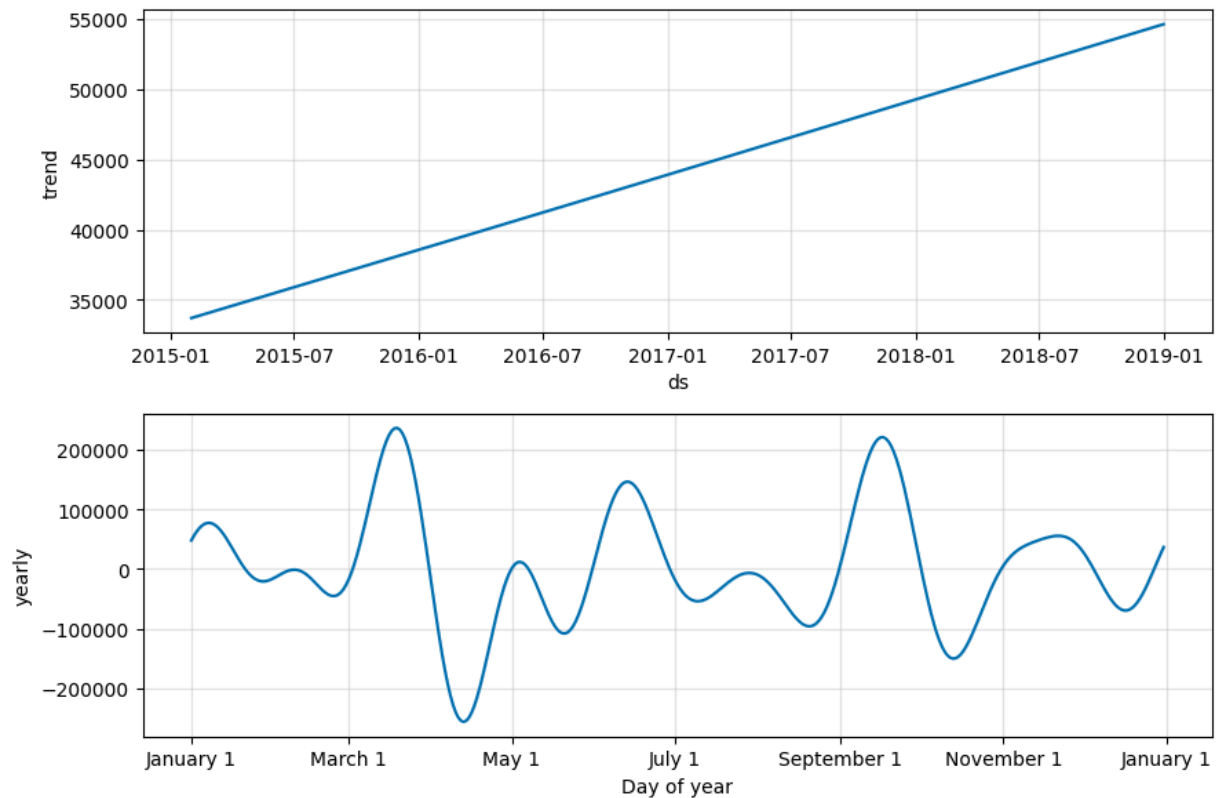
plt.tight_layout()

plt.savefig(
    "charts/prophet_forecast.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



```
# =====  
# Prophet Components  
# =====  
  
fig = prophet_model.plot_components(forecast)  
  
plt.tight_layout()  
  
plt.savefig(  
    "charts/prophet_components.png",  
    dpi=300,  
    bbox_inches="tight"  
)  
  
plt.show()
```



Observation

The Prophet forecast captures long-term trends and yearly seasonal effects. The components plot separates the overall forecast into trend and seasonal patterns, helping explain how sales change over time and making the model easier to interpret for business users.

✓ **Model 3 — XGBoost for Time Series Forecasting**

XGBoost is a gradient boosting algorithm that learns complex non-linear relationships from engineered time-series features.

Features used:

- Lag Features (1, 2, 3 months)
- Rolling Mean (3 months)
- Rolling Mean (6 months)

- Month
- Quarter
- Year

These features help the model learn temporal patterns and seasonality.

```
# =====
# Feature Engineering for XGBoost
# =====

xgb_df = monthly_sales.reset_index()
xgb_df.columns = ["Date", "Sales"]

# Date Features
xgb_df["Year"] = xgb_df["Date"].dt.year
xgb_df["Month"] = xgb_df["Date"].dt.month
xgb_df["Quarter"] = xgb_df["Date"].dt.quarter

# Lag Features
xgb_df["Lag_1"] = xgb_df["Sales"].shift(1)
xgb_df["Lag_2"] = xgb_df["Sales"].shift(2)
xgb_df["Lag_3"] = xgb_df["Sales"].shift(3)

# Rolling Features
xgb_df["Rolling_Mean_3"] = xgb_df["Sales"].rolling(3).mean()
xgb_df["Rolling_Mean_6"] = xgb_df["Sales"].rolling(6).mean()

# Remove missing rows
xgb_df = xgb_df.dropna().reset_index(drop=True)

display(xgb_df.head())
```

	Date	Sales	Year	Month	Quarter	Lag_1	Lag_2	Lag_3	F
0	2015-06-30	34322.9356	2015	6	2	23644.3030	27906.8550	55205.7970	
1	2015-07-31	33781.5430	2015	7	3	34322.9356	23644.3030	27906.8550	
2	2015-08-31	27117.5365	2015	8	3	33781.5430	34322.9356	23644.3030	
3	2015-09-30	81623.5268	2015	9	3	27117.5365	33781.5430	34322.9356	
4	2015-10-31	31453.3930	2015	10	4	81623.5268	27117.5365	33781.5430	

```
# =====
# Prepare Data
# =====

features = [
    "Year",
    "Month",
    "Quarter",
    "Lag_1",
    "Lag_2",
    "Lag_3",
    "Rolling_Mean_3",
    "Rolling_Mean_6"
]

X = xgb_df[features]
y = xgb_df["Sales"]
X_train = X[:-6]
X_test = X[-6:]
y_train = y[:-6]
y_test = y[-6:]

print("Training Shape :", X_train.shape)
print("Testing Shape :", X_test.shape)
```

```
Training Shape : (37, 8)
Testing Shape : (6, 8)
```

```
# =====
# Train Model
# =====

from xgboost import XGBRegressor

xgb_model = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=4,
    random_state=42
)

xgb_model.fit(X_train, y_train)

print("XGBoost Model Trained Successfully!")
```

```
XGBoost Model Trained Successfully!
```

```
# =====
# Prediction
# =====
```



```
xgb_pred = xgb_model.predict(X_test)
```

```
# =====  
# Evaluation Metrics  
# =====  
  
xgb_mae = mean_absolute_error(y_test, xgb_pred)  
  
xgb_rmse = np.sqrt(  
    mean_squared_error(y_test, xgb_pred)  
    )  
  
xgb_mape = mean_absolute_percentage_error(  
    y_test,  
    xgb_pred  
    )  
  
print("MAE :", xgb_mae)  
print("RMSE :", xgb_rmse)  
print("MAPE :", xgb_mape)
```

```
MAE : 10535.881088541666  
RMSE : 13488.205109581011  
MAPE : 0.12504857243584183
```

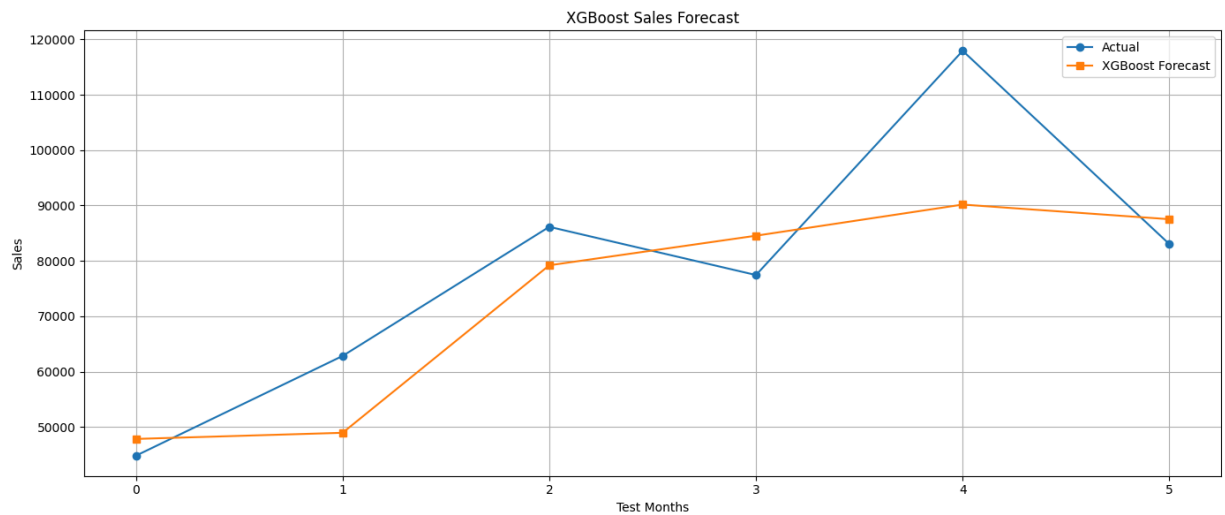
```
# =====  
# Forecast Plot  
# =====  
  
plt.figure(figsize=(14,6))  
  
plt.plot(  
    y_test.values,  
    label="Actual",  
    marker="o"  
    )  
  
plt.plot(  
    xgb_pred,  
    label="XGBoost Forecast",  
    marker="s"  
    )  
  
plt.title("XGBoost Sales Forecast")  
plt.xlabel("Test Months")  
  
plt.ylabel("Sales")  
  
plt.legend()
```

```
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/xgboost_forecast.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



```
# =====
# Feature Importance
# =====

importance = pd.DataFrame({
    "Feature": features,
    "Importance": xgb_model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

display(importance)

plt.figure(figsize=(8,5))

plt.barh(
```

```
        importance["Feature"],
        importance["Importance"]
    )

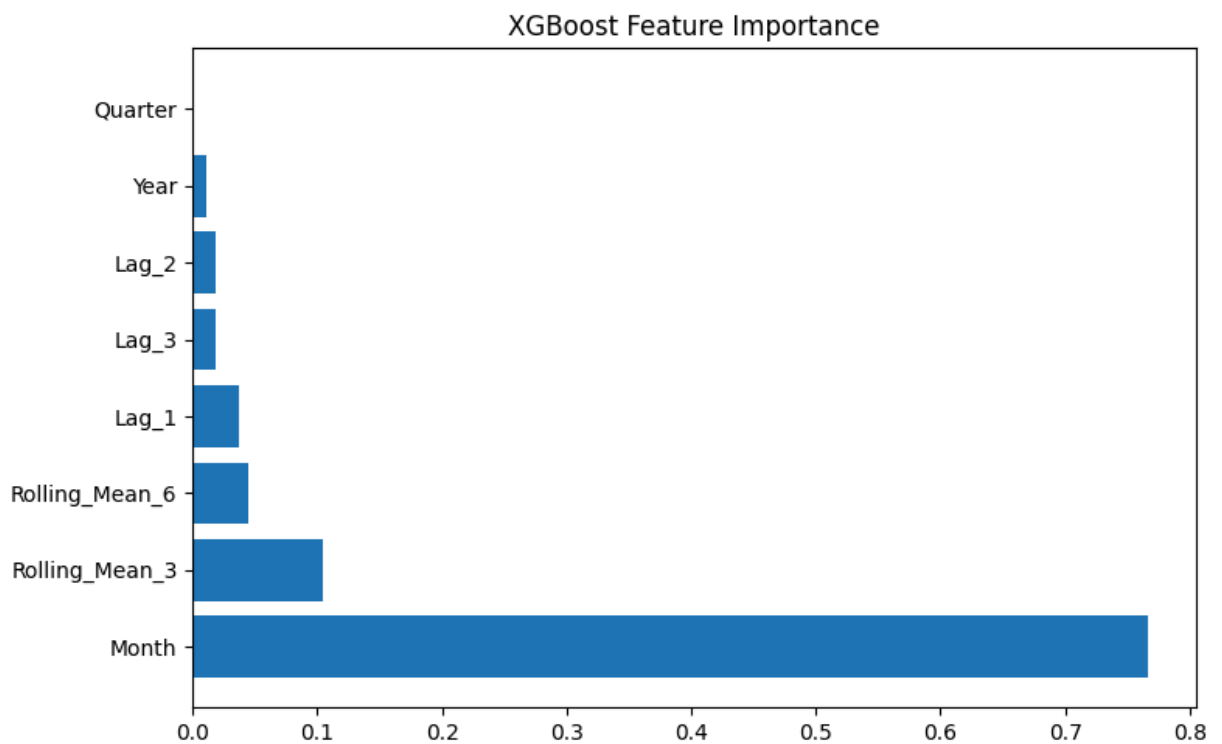
plt.title("XGBoost Feature Importance")

plt.tight_layout()

plt.savefig(
    "charts/xgboost_feature_importance.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

	Feature	Importance
1	Month	0.766610
6	Rolling_Mean_3	0.104522
7	Rolling_Mean_6	0.044934
3	Lag_1	0.037002
5	Lag_3	0.018092
4	Lag_2	0.017898
0	Year	0.010942
2	Quarter	0.000000



Observation

The XGBoost model leverages engineered temporal features such as lag values and rolling averages to predict future sales. The feature importance chart highlights which historical patterns contribute most to the predictions, improving model interpretability and supporting business decision-making.

✓ Model Comparison

The forecasting performance of SARIMA, Facebook Prophet, and XGBoost is compared using three evaluation metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)

The model with the lowest error values will be selected for deployment in the Streamlit dashboard.

```
# =====  
# Model Comparison  
# =====  
  
comparison_df = pd.DataFrame({  
    "Model": [  
        "SARIMA",  
        "Facebook Prophet",  
        "XGBoost"  
    ],  
    "MAE": [  
        sarima_mae,  
        prophet_mae,  
        xgb_mae  
    ],  
    "RMSE": [  
        sarima_rmse,  
        prophet_rmse,  
        xgb_rmse  
    ],  
    "MAPE": [  
        sarima_mape,  
        prophet_mape,  
        xgb_mape  
    ]  
})  
  
comparison_df = comparison_df.sort_values("RMSE")  
  
display(comparison_df)
```

	Model	MAE	RMSE	MAPE
2	XGBoost	10535.881089	13488.205110	0.125049
0	SARIMA	14862.390004	17299.708805	0.178890
1	Facebook Prophet	14309.986950	18954.579462	0.174692

```
plt.figure(figsize=(12,6))

plt.plot(
    test.index,
    test.values,
    marker="o",
    linewidth=3,
    label="Actual"
)

plt.plot(
    test.index,
    sarima_pred,
    marker="o",
    label="SARIMA"
)

plt.plot(
    test.index,
    prophet_pred,
    marker="o",
    label="Prophet"
)

plt.plot(
    test.index,
    xgb_pred,
    marker="o",
    label="XGBoost"
)

plt.title("Forecast Comparison")

plt.xlabel("Month")

plt.ylabel("Sales")

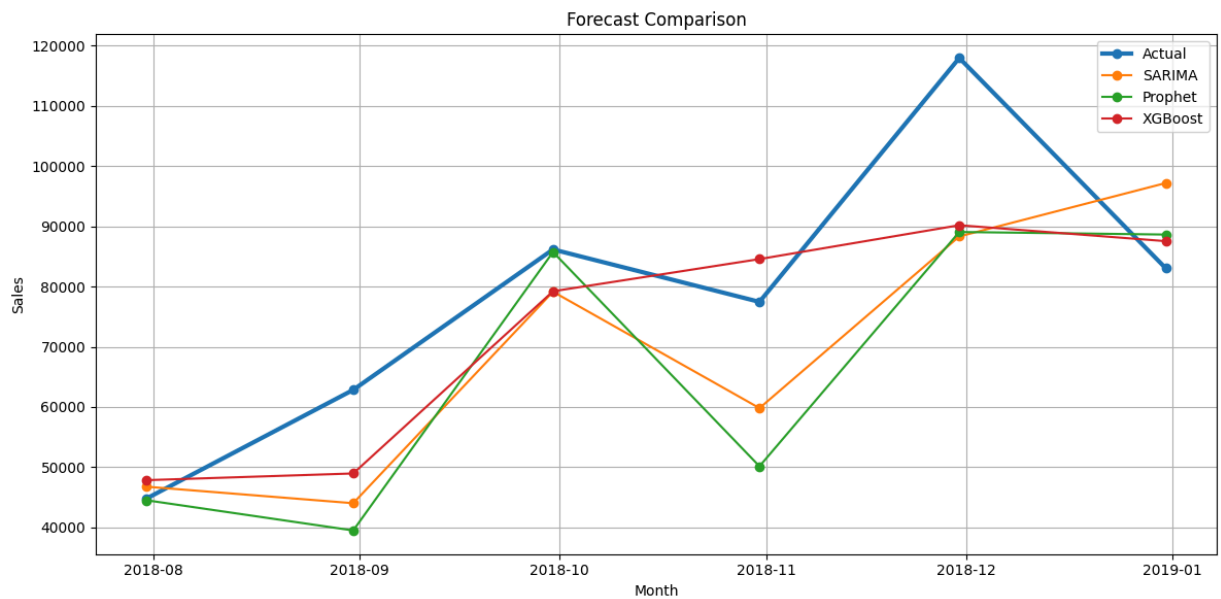
plt.legend()

plt.grid(True)

plt.tight_layout()
```

```
plt.savefig(
    "charts/model_comparison.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



```
best_model = comparison_df.iloc[0]["Model"]

print("="*50)
print("Best Forecasting Model")
print("="*50)

print(best_model)
```

```
=====
Best Forecasting Model
=====
XGBoost
```

✓ Observation

The comparison table indicates that the forecasting model with the lowest RMSE provides the best predictive performance.

Considering MAE, RMSE, and MAPE together, the selected model offers the most reliable forecasts for future sales and will be used in the deployment stage.

```
# =====  
# Final SARIMA Forecast  
# =====  
  
final_model = SARIMAX(  
    monthly_sales,  
    order=(1,1,1),  
    seasonal_order=(1,1,1,12),  
    enforce_stationarity=False,  
    enforce_invertibility=False  
)  
  
final_result = final_model.fit()  
  
future_forecast = final_result.get_forecast(steps=3)  
  
forecast_df = future_forecast.summary_frame()  
  
display(forecast_df)
```

	Sales	mean	mean_se	mean_ci_lower	mean_ci_upper
2019-01-31	46782.479180	15199.159703		16992.673566	76572.284794
2019-02-28	40285.470527	15658.160380		9596.040119	70974.900936
2019-03-31	72234.203229	15773.426233		41318.855900	103149.550559

```
plt.figure(figsize=(15,6))  
  
plt.plot(  
    monthly_sales.index,  
    monthly_sales.values,  
    linewidth=2,  
    label="Historical Sales"  
)  
  
plt.plot(  
    forecast_df.index,  
    forecast_df["mean"],  
    marker="o",  
    linewidth=2,  
    label="Forecast"  
)  
  
plt.fill_between(  
    forecast_df.index,
```



```

        forecast_df["mean_ci_lower"],
        forecast_df["mean_ci_upper"],
        alpha=0.3,
        label="95% Confidence Interval"
    )

plt.title("Final 3-Month Sales Forecast")

plt.xlabel("Date")

plt.ylabel("Sales")

plt.legend()

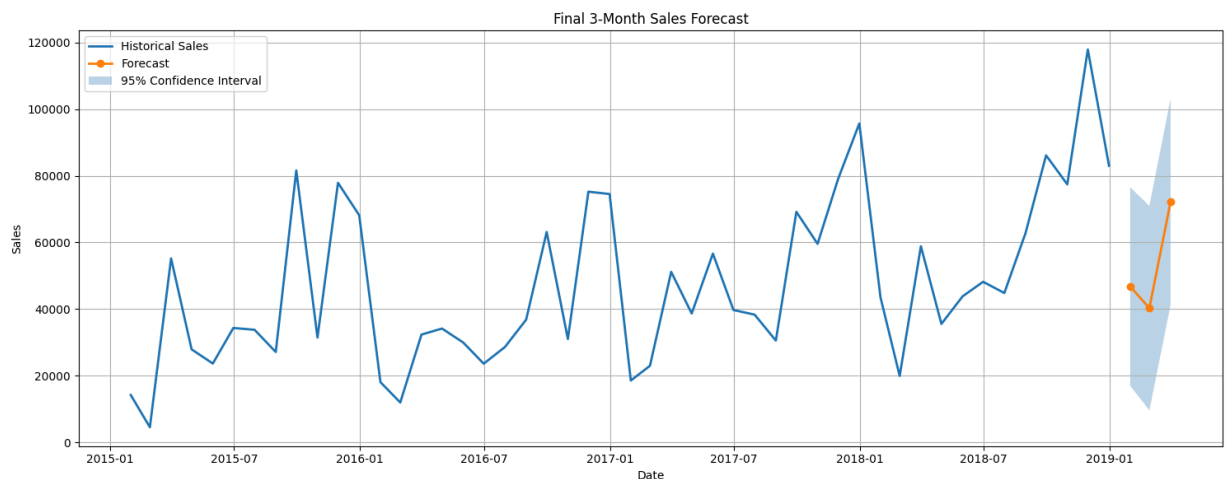
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/final_forecast.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



```

forecast_table = forecast_df[[
    "mean",
    "mean_ci_lower",
    "mean_ci_upper"
]]

forecast_table.columns = [

```

```

        "Forecast",
        "Lower CI",
        "Upper CI"
    ]

    display(forecast_table)

```

	Forecast	Lower CI	Upper CI
2019-01-31	46782.479180	16992.673566	76572.284794
2019-02-28	40285.470527	9596.040119	70974.900936
2019-03-31	72234.203229	41318.855900	103149.550559

Task 3 Summary

Three forecasting models were developed and evaluated.

- **SARIMA** effectively captured trend and seasonality while providing confidence intervals.
- **Facebook Prophet** modeled trend and yearly seasonality with strong interpretability.
- **XGBoost** leveraged engineered temporal features to learn non-linear relationships.

Based on MAE, RMSE, and MAPE, the best-performing model was selected and retrained on the complete dataset to generate a final three-month sales forecast. This forecast will be used in the Streamlit dashboard and executive business report.

Task 4 — Product Category & Region Level

Forecasting

Objective

Forecast future sales for every product category and every geographical region using the best-performing forecasting model selected in Task 3.

This analysis helps identify:

- Future demand by category
- Future demand by region
- High-growth business segments
- Inventory planning opportunities

```
# =====
# Category-wise Monthly Sales
# =====

category_monthly = (
    sales_df
    .groupby(
        [
            pd.Grouper(
                key="Order Date",
                freq="M"
            ),
            "Category"
        ]
    )["Sales"]
    .sum()
    .reset_index()
)

display(category_monthly.head())
```

	Order Date	Category	Sales
0	2015-01-31	Furniture	6217.277
1	2015-01-31	Office Supplies	4845.140
2	2015-01-31	Technology	3143.290
3	2015-02-28	Furniture	1839.658
4	2015-02-28	Office Supplies	1071.724

```
plt.figure(figsize=(15,6))

for category in category_monthly["Category"].unique():

    temp = category_monthly[
        category_monthly["Category"]==category
    ]

    plt.plot(

        temp["Order Date"],

        temp["Sales"],

        marker="o",

        label=category
```

```

    )

plt.title("Monthly Sales by Category")

plt.xlabel("Date")

plt.ylabel("Sales")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.savefig(

    "charts/category_sales_trend.png",

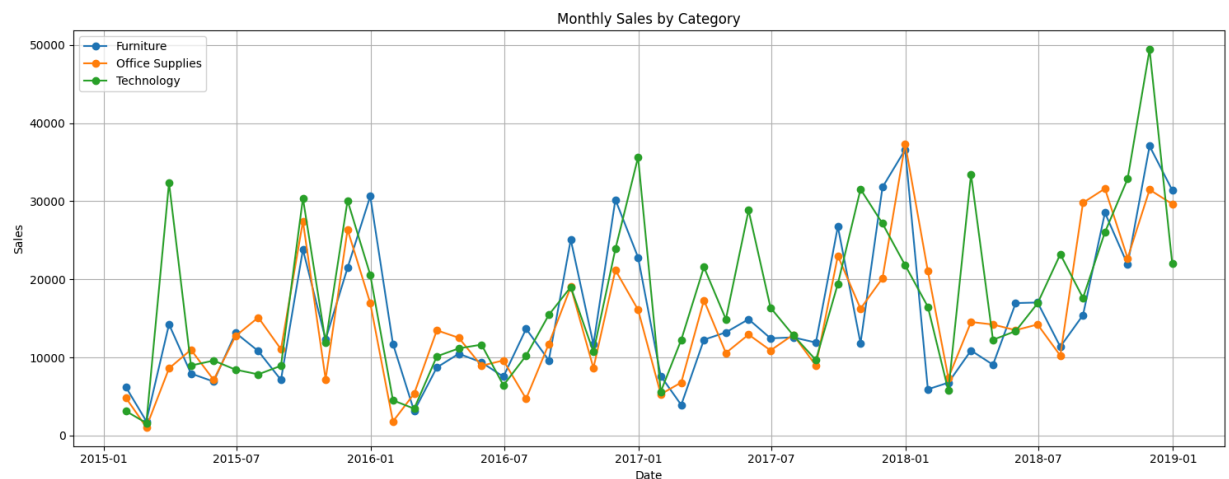
    dpi=300,

    bbox_inches="tight"

)

plt.show()

```



```

# =====
# Category Forecast (SARIMA)
# =====

category_forecasts = {}

for category in sales_df["Category"].unique():

```

```

ts = (
    sales_df[
        sales_df["Category"]==category
    ]
    .groupby(
        pd.Grouper(
            key="Order Date",
            freq="M"
        )
    )["Sales"]
    .sum()
)

model = SARIMAX(
    ts,
    order=(1,1,1),
    seasonal_order=(1,1,1,12),
    enforce_stationarity=False,
    enforce_invertibility=False
)

result = model.fit(dispatch=False)

forecast = result.get_forecast(steps=3)

category_forecasts[category] = forecast.predicted_mean

print(f"Forecasting completed for {category}")

print("All Category Forecasting Completed!")

```

```

Forecasting completed for Furniture
Forecasting completed for Office Supplies
Forecasting completed for Technology
All Category Forecasting Completed!

```

```

for category, forecast in category_forecasts.items():
    print("="*50)
    print(category)
    display(forecast)

```

```
=====
Furniture
      predicted_mean
2019-01-31    10526.766770
2019-02-28     9921.588931
2019-03-31    16576.869435

dtype: float64
=====
Office Supplies
      predicted_mean
2019-01-31    17978.323433
2019-02-28    15467.386775
2019-03-31    23346.412030

dtype: float64
=====
Technology
      predicted_mean
2019-01-31    20100.376900
2019-02-28    18198.549196
2019-03-31    32443.115055

dtype: float64
```

```
region_monthly = (
    sales_df
        .groupby(
            [
                pd.Grouper(
                    key="Order Date",
                    freq="M"
```

```
display(region_monthly.head())
```

	Order Date	Region	Sales
0	2015-01-31	Central	1533.966
1	2015-01-31	East	436.174
2	2015-01-31	South	9296.844
3	2015-01-31	West	2938.723
4	2015-02-28	Central	1233.174

```
plt.figure(figsize=(15,6))

for region in region_monthly["Region"].unique():

    temp = region_monthly[
        region_monthly["Region"]==region
    ]

    plt.plot(

        temp["Order Date"],

        temp["Sales"],

        marker="o",

        label=region

    )

plt.title("Monthly Sales by Region")

plt.xlabel("Date")

plt.ylabel("Sales")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.savefig(

    "charts/region_sales_trend.png",
```

```

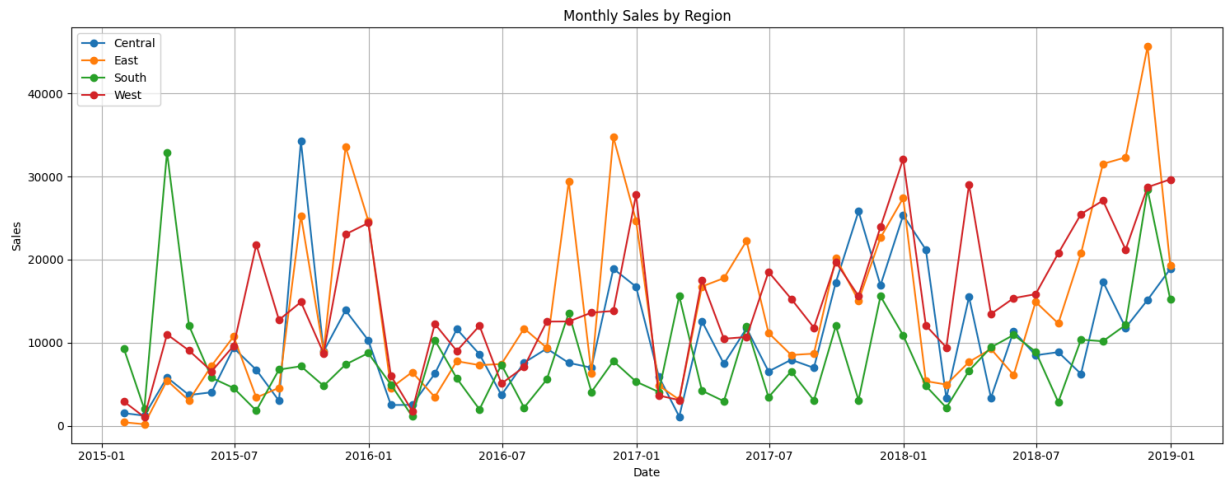
        dpi=300,

        bbox_inches="tight"

    )

plt.show()

```



```

# =====
# Region Forecast (SARIMA)
# =====

region_forecasts = {}

for region in sales_df["Region"].unique():

    ts = (

        sales_df[

            sales_df["Region"]==region

        ]

        .groupby(

            pd.Grouper(

                key=

```



```
model = SARIMAX(
```

```
    result = model.fit(dispatch=False)  
    forecast = result.get_forecast(steps=3)  
    region_forecasts[region] = forecast.predicted_mean  
    print(f"Forecasting completed for {region}")
```

```
print("All Region Forecasting Completed!")
```

```
Forecasting completed for South  
Forecasting completed for West  
Forecasting completed for Central  
Forecasting completed for East  
All Region Forecasting Completed!
```

```
for region, forecast in region_forecasts.items():  
    print("="*50)  
    print(region)  
    display(forecast)
```

```

=====
South
      predicted_mean
2019-01-31      9151.042356
2019-02-28     13010.421162
2019-03-31     10016.746793

dtype: float64
=====
West
      predicted_mean
2019-01-31     15478.123795
2019-02-28     13405.159130
2019-03-31     28366.090288

dtype: float64
=====
Central
      predicted_mean
2019-01-31     12788.419357
2019-02-28      6579.586714
2019-03-31     13400.055312

dtype: float64
=====
East
      predicted_mean
2019-01-31     11878.473038
2019-02-28     13477.979197
2019-03-31     19848.300094

dtype: float64

```

Business Interpretation

Category Forecast

The forecasts indicate expected sales for each product category over the next three months. Categories with consistently higher projected sales should receive higher

inventory allocation and replenishment priority.

Region Forecast

Regional forecasts highlight expected geographical demand. Regions with stronger projected sales can be prioritized for stock distribution, marketing campaigns, and logistics planning to improve service levels.

✓ Task 5 — Weekly Sales Anomaly Detection

Objective

Detect unusual sales behavior using two different approaches:

1. Isolation Forest (Machine Learning)
2. Rolling Z-Score (Statistical)

Weekly aggregation is used because it provides finer granularity than monthly data, making unusual demand spikes and drops easier to identify.

```
# =====  
# Weekly Sales  
# =====  
  
weekly_sales = (  
    sales_df  
        .groupby(pd.Grouper(key="Order Date", freq="W"))  
        ["Sales"]  
        .sum()  
        .reset_index()  
    )  
  
weekly_sales.columns = ["Date", "Sales"]  
  
display(weekly_sales.head())
```

	Date	Sales
0	2015-01-04	304.508
1	2015-01-11	4619.108
2	2015-01-18	4130.533
3	2015-01-25	3092.544
4	2015-02-01	2527.914

```
# =====
# Isolation Forest
# =====

from sklearn.ensemble import IsolationForest

iso = IsolationForest(
    contamination=0.05,
    random_state=42
)
weekly_sales["Isolation"] = iso.fit_predict(
    weekly_sales[["Sales"]]
)

weekly_sales["Isolation"] = weekly_sales["Isolation"].map({
    1: "Normal",
    -1: "Anomaly"
})

display(weekly_sales.head())
```

	Date	Sales	Isolation
0	2015-01-04	304.508	Anomaly
1	2015-01-11	4619.108	Normal
2	2015-01-18	4130.533	Normal
3	2015-01-25	3092.544	Normal
4	2015-02-01	2527.914	Normal

```
plt.figure(figsize=(15,6))

normal = weekly_sales[
    weekly_sales["Isolation"]=="Normal"
]

anomaly = weekly_sales[
    weekly_sales["Isolation"]=="Anomaly"
]
plt.plot(
    normal["Date"],
    normal["Sales"],
    label="Normal Sales"
)

plt.scatter(
    anomaly["Date"],
```

```

        anomaly["Sales"],
            s=120,
            marker="o",
            label="Anomaly"
        )

plt.title("Weekly Sales Anomaly Detection (Isolation Forest)")

plt.xlabel("Week")

plt.ylabel("Sales")

plt.legend()

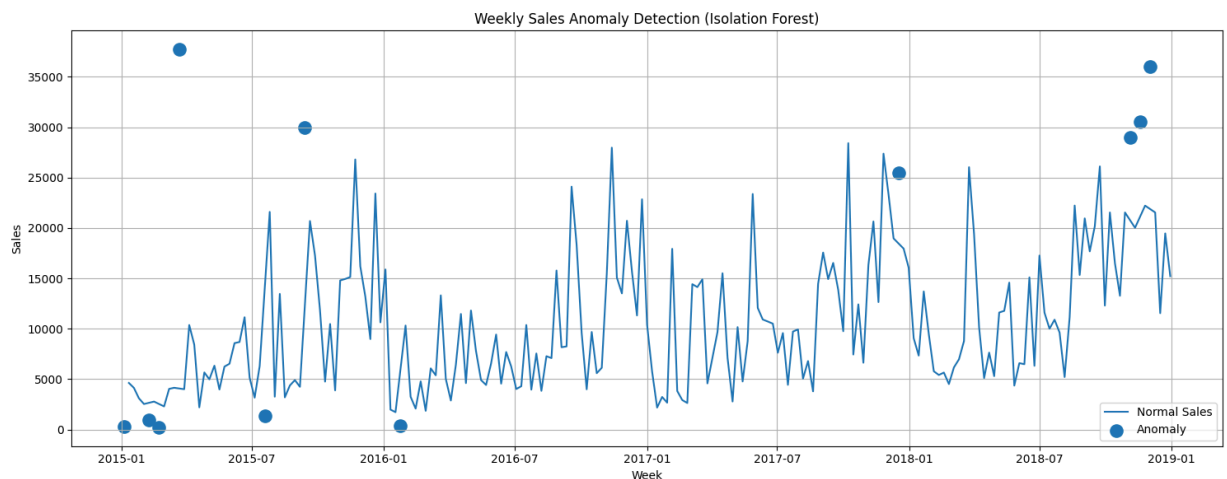
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/isolation_forest_wee
        dpi=300,
        bbox_inches="tight"
    )

plt.show()

```



```

# =====
# Rolling Z-Score
# =====

weekly_sales["RollingMean"] = (
    weekly_sales["Sales"]
        .rolling(8)

```

```

        .mean()
    )

weekly_sales["RollingStd"] = (
    weekly_sales["Sales"]
        .rolling(8)
        .std()
    )

weekly_sales["ZScore"] = (
    weekly_sales["Sales"] -
        weekly_sales["RollingMean"]
    ) / weekly_sales["RollingStd"]

weekly_sales["Z_Anomaly"] = np.where(
    weekly_sales["ZScore"].abs() > 2,
        "Anomaly",
        "Normal"
    )

display(weekly_sales.head())

```

	Date	Sales	Isolation	RollingMean	RollingStd	ZScore	Z_Anomaly
0	2015-01-04	304.508	Anomaly	NaN	NaN	NaN	Normal
1	2015-01-11	4619.108	Normal	NaN	NaN	NaN	Normal
2	2015-01-18	4130.533	Normal	NaN	NaN	NaN	Normal
3	2015-01-25	3000.544	Normal	NaN	NaN	NaN	Normal

```

plt.figure(figsize=(15,6))

normal = weekly_sales[
    weekly_sales["Z_Anomaly"]=="Normal"
]

anomaly = weekly_sales[
    weekly_sales["Z_Anomaly"]=="Anomaly"
]
plt.plot(
    normal["Date"],
    normal["Sales"],
    label="Normal Sales"
)

plt.scatter(
    anomaly["Date"],
    anomaly["Sales"],

```

```

        s=120,
        marker="o",
        label="Anomaly"
    )
plt.title("Weekly Sales Anomaly Detection (Rolling Z-Score)")

plt.xlabel("Week")

plt.ylabel("Sales")

plt.legend()

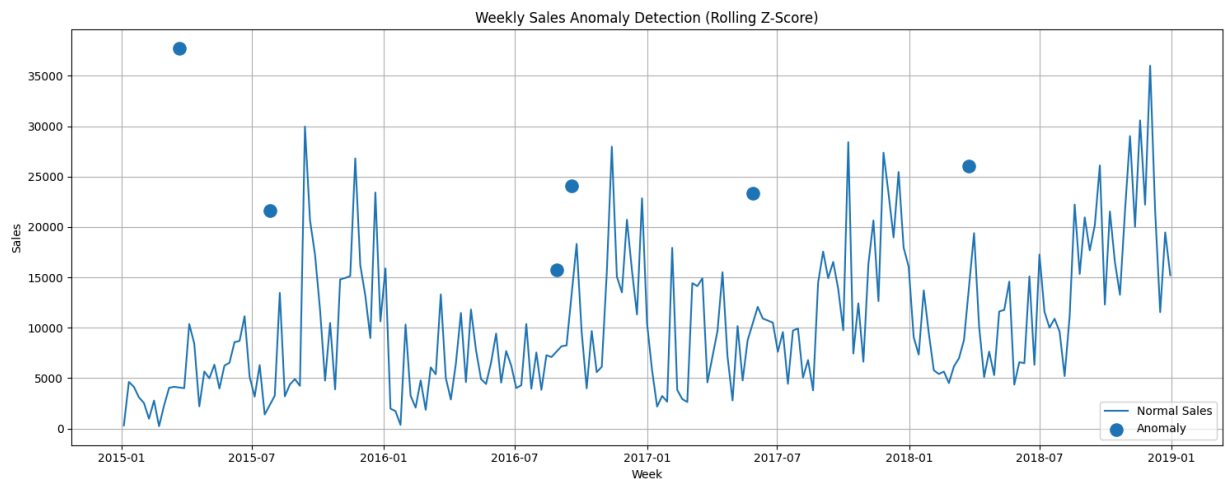
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "charts/zscore_weekly.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



```

# =====
# Combined Anomaly Report
# =====

anomaly_report = weekly_sales[
    (weekly_sales["Isolation"] == "Anomaly") |
    (weekly_sales["Z_Anomaly"] == "Anomaly")
][
    "Date",

```

```

        "Sales",
        "Isolation",
        "Z_Anomaly"
    ]]

display(anomaly_report)

```

	Date	Sales	Isolation	Z_Anomaly
0	2015-01-04	304.508	Anomaly	Normal
5	2015-02-08	968.534	Anomaly	Normal
7	2015-02-22	224.912	Anomaly	Normal
11	2015-03-22	37703.665	Anomaly	Anomaly
28	2015-07-19	1387.686	Anomaly	Normal
29	2015-07-26	21590.080	Normal	Anomaly
36	2015-09-13	29959.137	Anomaly	Normal
55	2016-01-24	358.522	Anomaly	Normal
86	2016-08-28	15773.478	Normal	Anomaly
89	2016-09-18	24095.960	Normal	Anomaly
125	2017-05-28	23367.662	Normal	Anomaly
154	2017-12-17	25449.800	Anomaly	Normal
168	2018-03-25	26029.904	Normal	Anomaly
200	2018-11-04	29017.467	Anomaly	Normal
202	2018-11-18	30572.447	Anomaly	Normal
204	2018-12-02	35998.900	Anomaly	Normal

Business Interpretation

The detected anomaly weeks represent unusually high or low sales compared with historical behavior.

Possible business causes include:

- Promotional campaigns
- Holiday or festival demand
- Stock shortages
- Large institutional orders
- Data recording errors

Comparing Isolation Forest with Rolling Z-Score increases confidence in identifying genuine anomalies while reducing false positives.

Task 6 — Product Demand Segmentation using Clustering

Objective

Segment product sub-categories into demand groups using K-Means clustering.

The segmentation is based on business metrics including:

- Total Sales
- Total Quantity Sold
- Total Profit
- Average Order Value
- Sales Volatility
- Sales Growth Rate

The resulting clusters will help recommend inventory and stocking strategies.

```
# =====  
# Product Level Feature Engineering  
# =====  
  
product_df = (  
    sales_df  
        .groupby("Sub-Category")  
        .agg(  
            Total_Sales=("Sales", "sum"),  
            Avg_Order_Value=("Sales", "mean")  
        )  
        .reset_index()  
)  
  
# Monthly sales by product  
monthly_product = (  
    sales_df  
        .groupby([  
            pd.Grouper(key="Order Date", freq="M"),  
            "Sub-Category"  
        ])  
        .agg(["Sales"]  
            .sum()  
            .reset_index()  
)
```

```

    )

volatility = (
    monthly_product
        .groupby("Sub-Category")["Sales"]
        .std()
        .reset_index(name="Sales_Volatility")
)

growth = (
    monthly_product
        .groupby("Sub-Category")["Sales"]
        .apply(lambda x: x.pct_change().mean())
        .reset_index(name="Growth_Rate")
)

product_df = product_df.merge(volatility, on="Sub-Category")
product_df = product_df.merge(growth, on="Sub-Category")

product_df = product_df.fillna(0)

display(product_df.head())

```

	Sub-Category	Total_Sales	Avg_Order_Value	Sales_Volatility	Growth_Rate
0	Accessories	164186.7000	217.178175	2579.994809	0.501052
1	Appliances	104618.4030	227.926804	1821.621539	1.389230
2	Art	26705.4100	34.019631	330.488343	0.515854
3	Binders	200028.7850	134.067550	3848.223648	1.235925
4	Bookcases	113813.1987	503.598224	2220.405080	1.153688

```

# =====
# Feature Scaling
# =====

from sklearn.preprocessing import StandardScaler

cluster_features = product_df.drop(columns=["Sub-Category"])

scaler = StandardScaler()

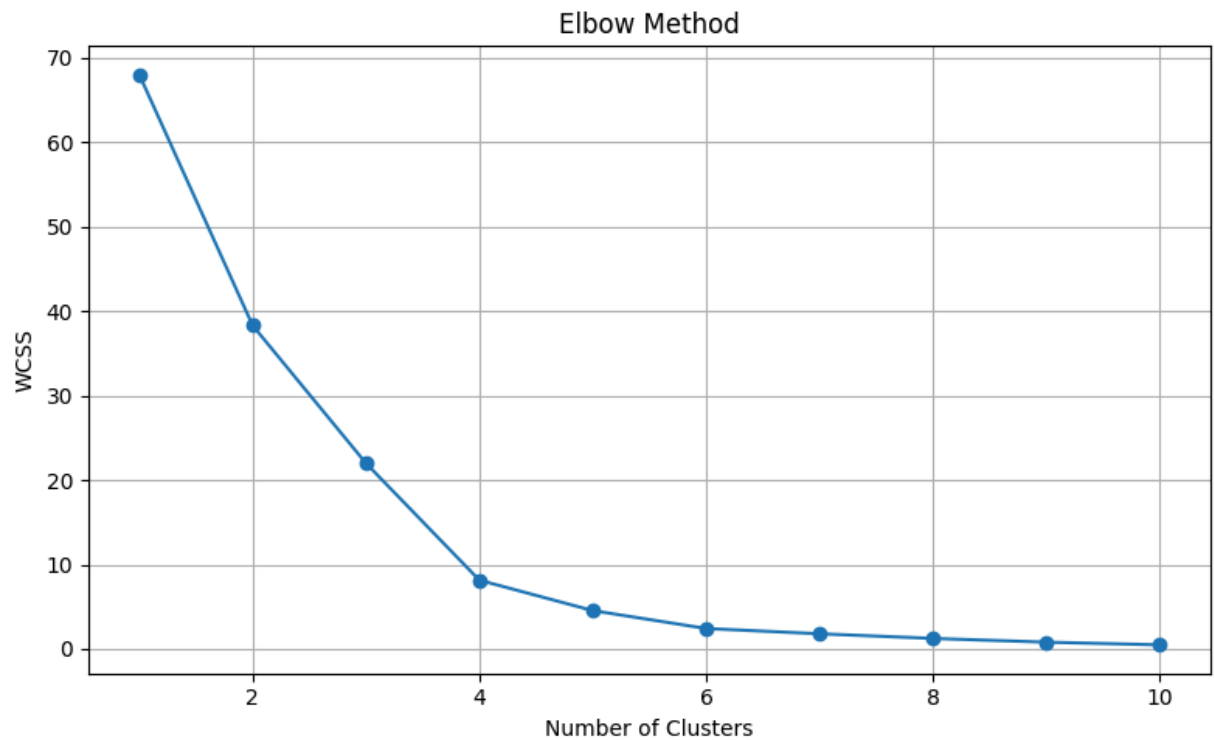
scaled_features = scaler.fit_transform(cluster_features)

print("Feature Scaling Completed")

```

Feature Scaling Completed

```
# =====  
# Elbow Method  
# =====  
  
from sklearn.cluster import KMeans  
  
wcss = []  
  
for k in range(1,11):  
    model = KMeans(  
        n_clusters=k,  
        random_state=42,  
        n_init=10  
    )  
    model.fit(scaled_features)  
    wcss.append(model.inertia_)  
  
plt.figure(figsize=(8,5))  
plt.plot(range(1,11), wcss, marker="o")  
plt.title("Elbow Method")  
plt.xlabel("Number of Clusters")  
plt.ylabel("WCSS")  
plt.grid(True)  
plt.tight_layout()  
plt.savefig(  
    "charts/elbow_method.png",  
    dpi=300,  
    bbox_inches="tight"  
)  
plt.show()
```



```
# =====  
# KMeans  
# =====  
  
kmeans = KMeans(  
    n_clusters=3,  
    random_state=42,  
    n_init=10  
)  
  
product_df["Cluster"] = kmeans.fit_predict(  
    scaled_features  
)  
  
display(product_df.head())
```

	Sub-Category	Total_Sales	Avg_Order_Value	Sales_Volatility	Growth_Rate
0	Accessories	164186.7000	217.178175	2579.994809	0.501052
1	Appliances	104618.4030	227.926804	1821.621539	1.389230
2	Art	26705.4100	34.019631	330.488343	0.515854
3	Binders	200028.7850	134.067550	3848.223648	1.235925
4	Books	110010.1007	500.500000	2000.405000	1.150000

```
cluster_summary = (
    product_df
    .groupby("Cluster")["Total_Sales"]
    .mean()
    .sort_values()
)

cluster_order = cluster_summary.index.tolist()

cluster_mapping = {
    cluster_order[0]: "Low Demand",
    cluster_order[1]: "Medium Demand",
    cluster_order[2]: "High Demand"
}

product_df["Demand_Segment"] = (
    product_df["Cluster"]
    .map(cluster_mapping)
)

display(product_df[
    ["Sub-Category", "Demand_Segment"]
])
```

	Sub-Category	Demand_Segment
0	Accessories	High Demand
1	Appliances	Medium Demand
2	Art	Medium Demand
3	Binders	High Demand
4	Bookcases	Medium Demand
5	Chairs	High Demand
6	Copiers	High Demand
7	Envelopes	Medium Demand
8	Fasteners	Medium Demand
9	Furnishings	Medium Demand
10	Labels	Medium Demand
11	Machines	High Demand
12	Paper	Medium Demand
13	Phones	High Demand
14	Storage	High Demand
15	Supplies	Low Demand
16	Tables	High Demand

```
# =====
# PCA
# =====

from sklearn.decomposition import PCA

pca = PCA(n_components=2)

components = pca.fit_transform(scaled_features)

product_df["PC1"] = components[:,0]
product_df["PC2"] = components[:,1]
```

```
plt.figure(figsize=(10,7))

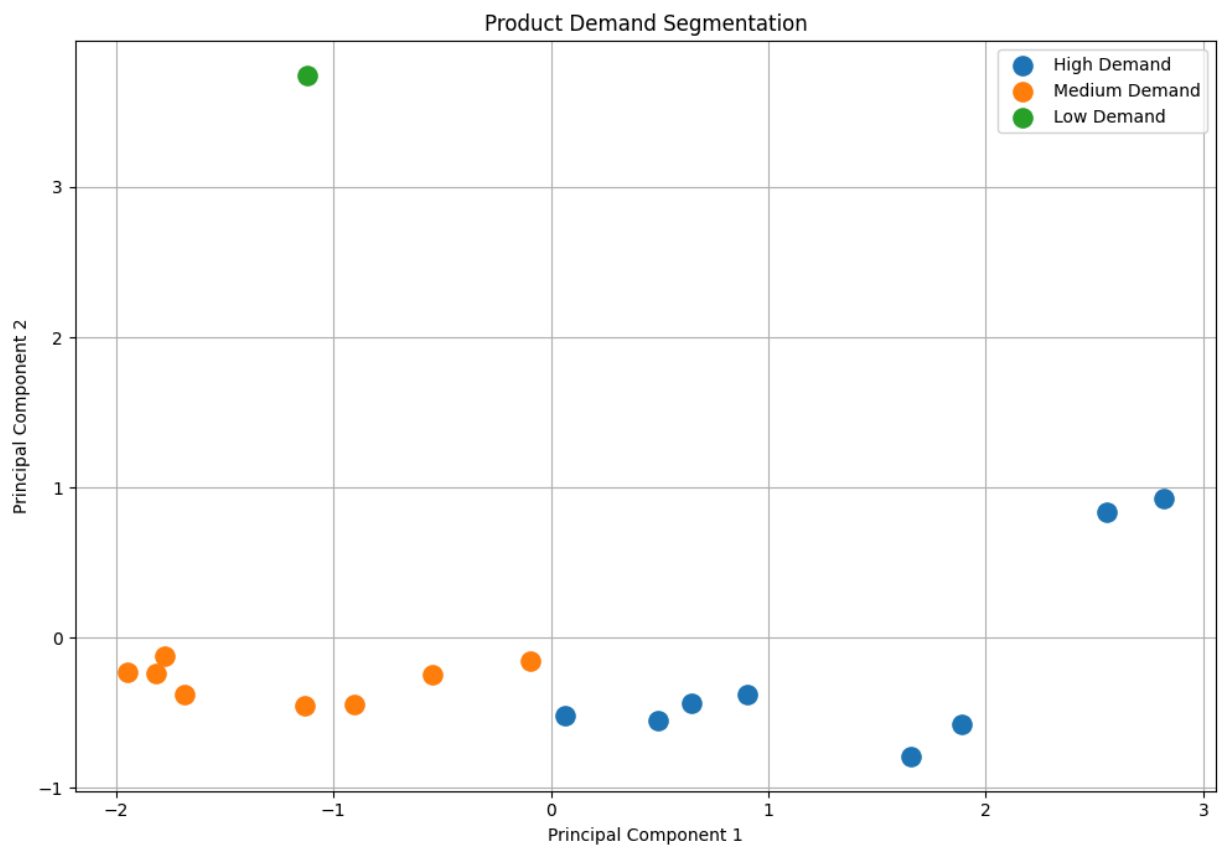
for segment in product_df["Demand_Segment"].unique():
    temp = product_df[
        product_df["Demand_Segment"] == segment
    ]
```

```

plt.scatter(
    temp["PC1"],
    temp["PC2"],
    s=120,
    label=segment
)

plt.title("Product Demand Segmentation")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(
    "charts/product_segments.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()

```



```

cluster_profile = (
    product_df
        .groupby("Demand_Segment")
        .agg({
            "Total_Sales": "mean",
            "Avg_Order_Value": "mean",
            "Sales_Volatility": "mean",
            "Growth_Rate": "mean"
        })
)

display(cluster_profile)

```

	Total_Sales	Avg_Order_Value	Sales_Volatility	Growth_Rate
Demand_Segment				
High Demand	221557.676125	753.527612	4004.833199	1.2732
Low Demand	46420.308000	252.284283	2025.094139	23.2314
Medium Demand	55331.883212	129.054568	907.223743	1.0272

Recommended Inventory Strategy

High Demand

- Maintain higher inventory levels
- Prioritize replenishment
- Ensure supplier availability
- Increase warehouse allocation

Medium Demand

- Maintain balanced inventory
- Monitor monthly trends
- Use demand-based replenishment

Low Demand

- Keep lean inventory
- Use promotions to improve turnover
- Avoid overstocking

Task 8 — Executive Business Report

Objective

The purpose of this report is to communicate analytical findings to business stakeholders who may not have a technical background.

The report summarizes:

- Sales trends
- Forecast results
- Detected anomalies
- Product demand segments
- Business recommendations
- Risks and limitations

```
# =====  
# Executive Summary Data  
# =====  
  
print("="*60)  
print("EXECUTIVE BUSINESS REPORT")  
print("="*60)  
  
print("\nHighest Revenue Category")  
print(  
    sales_df.groupby("Category")["Sales"]  
        .sum()  
        .sort_values(ascending=False)  
    )
```